PERSONAL AIR EXPOSURE COMPANION

Team build plan • MVP specification • Step-by-step execution guide

Purpose
Help people in highly polluted Indian cities understand where, when, and for how long they may encounter polluted air — then offer practical ways to reduce exposure.

Team
3 members with clear ownership across data and science, backend and integration, and frontend and user experience.

Hackathon goal
Build one credible end-to-end user journey, use a qualifying AWS open-source tool or deploy on AWS, and demonstrate the working product in three minutes.

Working principle

Build a small product that is honest about data quality and useful to a real person. Do not add features just to make the app look bigger. If a feature does not improve exposure understanding, help the user take action, satisfy the hackathon rules, or strengthen the demo, postpone it.

Prepared as a practical build guide. Verify current hackathon deadlines, AWS eligibility, service pricing, API access rules, and data availability before relying on them.

Contents

Section	What it covers
1. Project in one page	Goal, audience, promise, and boundaries
2. Team roles	Who owns what and how work is shared
3. MVP scope	Must-build, later, and out-of-scope features
4. User journey and screens	The exact flow to demonstrate
5. Data and exposure science	Sources, concepts, calculation, and limitations
6. Technical architecture	Suggested stack, flow, and folder structure
7. Step-by-step build plan	Milestones, dependencies, and completion checks
8. Team workflow	Git, communication, integration, and definition of done
9. Testing and validation	Calculation, API, UI, and data-quality tests
10. AWS and deployment	Qualifying usage, deployment options, and cost safety
11. Demo and judging strategy	Three-minute demo and evidence
12. Risks and distraction guardrails	What can go wrong and what to postpone
13. Final checklist	Submission readiness
14. Official references	Starting points for data and platform documentation

Suggested approach: read Sections 1–3 together, agree on ownership and scope, then start the data validation milestone before spending much time on UI polish.

1. Project in one page

This section gives every team member the same understanding of the product.

The problem

People can check their city's Air Quality Index (AQI), but that number does not explain how their own daily routine relates to pollution. A student may spend time at home, travel on a busy road, attend college, and exercise outdoors. The conditions may differ at each place, and the person may not know which part of the day deserves the most attention.

The product

The Personal Air Exposure Companion combines a user's places and time spent there with available air-quality observations or forecasts. It estimates exposure over the tracked period, explains where data is uncertain, and offers practical actions.

Core promise: "Don't just tell me how polluted my city is. Help me understand the pollution conditions across my day and what I can reasonably change."

Target user

- First users: students, office workers, and regular commuters in one selected polluted Indian city.
- Start with one city and a limited area where usable monitoring data exists. Expand only after the core method works.
- Manual entry must work. Location tracking is optional, not required.

What makes it different

The differentiator is not a map showing AQI. It is a personal timeline that connects time, place, available pollutant measurements, uncertainty, and an actionable next step. The app must never pretend that a distant station gives an exact measurement of what a person breathed indoors or on every street.

MVP success

- A user can enter a simple daily routine and saved places.
- The app retrieves real air-quality data where available and labels its source and timestamp.
- The app calculates a time-weighted PM2.5 average for periods with usable data.
- The app shows data coverage and uncertainty, and does not treat missing values as zero.
- The app explains the largest contributors and provides at least one useful recommendation.
- The complete flow works in a deployed or qualifying AWS-backed build and can be shown in a three-minute video.

2. Team roles and responsibilities

Three people should have clear ownership. Everyone can help, but every workstream needs one person who is responsible for finishing it.

Member	Primary ownership	Tasks	What they must deliver
Member A — Data & Exposure Science	Data source, data quality, exposure calculation	Check OpenAQ/CPCB/forecast coverage; test timestamps and units; build calculation rules; define missing-data handling; write calculation tests; document limitations.	A working data sample, a tested exposure calculation, data-quality notes, and example results.
Member B — Backend & Integration	API, data models, integration, AWS backend	Set up FastAPI; create API contracts; implement endpoints; connect the data provider; validate inputs; manage environment variables; help deploy the backend.	A backend that returns consistent air-quality and exposure responses, with error handling and a deployment path.
Member C — Frontend & Product Experience	User flow, screens, charts, usability, demo story	Build onboarding; daily dashboard; exposure timeline; places and recommendations screens; connect APIs; ensure mobile layout; prepare demo visuals and user flow.	A usable frontend connected to the backend, with clear labels for measured, forecast, and estimated data.

Shared responsibilities

- All three members agree on the MVP scope, user story, data limitations, and definition of done.
- Member A owns scientific correctness, but Members B and C review how values are displayed.
- Member B owns API contracts, but Members A and C test them with realistic examples.
- Member C owns the user experience, but the whole team reviews it against the judging criteria.
- All three members test the final build, rehearse the demo, and understand where AWS is used.

A simple rule for avoiding blocked work

Agree on the data format and API response before everyone starts coding independently. Member C can build the layout using a clearly labelled sample JSON response while Member A validates the data and Member B builds the API. When the API is ready, replace the sample response with the real one.

Suggested team check-in

When	What to discuss	Timebox
Start of work session	Each person names their goal and what they need from others.	5 minutes
Mid-session if needed	Resolve a blocked dependency; do not redesign the whole product.	5–10 minutes
End of session	Show what works, list bugs, agree on the next smallest tasks.	10 minutes

3. MVP scope: what to build and what to postpone

Must build for the first version

Feature	MVP behaviour	Why it matters
City and saved places	User chooses one city and adds home, college/office, or another place manually.	Makes the product personal without forcing GPS.
Air-quality data	Show available PM2.5, optional AQI, source, observation time, and data type.	Builds trust and gives the calculation an input.
Daily activity timeline	User enters activity, start/end time, and place; can edit entries.	Defines the time periods being summarized.
Exposure estimate	Calculate a time-weighted PM2.5 average over periods with usable data.	Core product differentiator.
Coverage and uncertainty	Show tracked time and how much of it has usable data; label estimates clearly.	Prevents false confidence.
Main contributors	Explain which periods had the highest available concentration and how duration affects the average.	Turns a number into an understandable story.
Recommendations	Use simple, evidence-based rules for practical actions.	Helps users decide what to do next.
Weekly view	Summarize available data across days; avoid comparisons when coverage is poor.	Shows patterns without overclaiming.

Build later, only if the core works

- Cleaner-route comparison, once route-level data is good enough to make a meaningful comparison.
- Push notifications and scheduled alerts.
- Indoor sensors or integration with home air monitors.
- Automatic location tracking, if privacy, battery, and data-matching concerns are addressed.
- More cities and more pollutant types.
- An AI explanation assistant, only if it uses verified data and does not invent health claims.

Do not build now

- A custom satellite-image model.
- A general-purpose medical chatbot or diagnosis feature.
- Diet plans, social feeds, gamification, leaderboards, or complex profile customization.
- Many AWS services that add cost and setup work but do not improve the demo.
- A claim that the app can precisely measure the exact pollution a person inhaled without suitable sensors.

Scope guardrail: if a feature does not improve the exposure estimate, help the user understand it, help them act, satisfy the event requirement, or make the demo clearer, move it to the later list.

4. User journey and screens

A judge should be able to understand the product without a long explanation.

The core journey

Step	User action	App response
1. Start	Select city and choose manual setup or optional location tracking.	Explain what the app tracks and what it does not know.
2. Add routine	Add places and enter activity times.	Show a simple day timeline that can be edited.
3. Load air data	Open the dashboard.	Fetch current or historical data available for relevant locations and show source/time.
4. Review exposure	Open My Exposure.	Show weighted average for covered time, tracked duration, coverage, and timeline.
5. Underst and	Review main contributors.	Explain which periods had higher concentrations and why duration matters.
6. Act	Open Recommendations.	Suggest a practical step supported by the available information.

Screen 1 — Dashboard

- Selected city and latest available air-quality conditions.
- Estimated average PM2.5 during tracked time, not a vague health score.
- How many hours were tracked and how many had usable data.
- One plain-language finding and one next action.
- A visible timestamp and data-source link.

Screen 2 — My Exposure

- Timeline by hour or activity, with concentration and duration.
- A simple chart showing available PM2.5 values across the day.
- Weighted-average calculation and the period it covers.
- A clear note if values are estimated or some periods are missing.

Screen 3 — My Places

- Add/edit/delete saved places and set a place type.
- Show nearby stations if station coordinates are available.
- Show distance to a station and last observation time.
- Do not claim a nearby station equals the air inside a home, office, or moving vehicle.

Screen 4 — Recommendations and Reports

- Short recommendations with a reason for each one.
- Weekly pattern only where there is enough usable data.
- No unsupported claims about exact health benefit or a guaranteed exposure reduction.

Text wireframe for the dashboard

Today • Kanpur
Latest available PM2.5: [value] µg/m³
Source: [provider/station] • Updated: [time]

My estimated average
[value] µg/m³ during [X] tracked hours
Usable data: [Y] of [X] hours

What stands out
[Time period] had the highest available PM2.5 estimate.

One action
[A practical recommendation with a short reason]

5. Data and exposure science

This is the most important trust issue in the product. Correctly explaining uncertainty is better than showing a false precise number.

Understand the terms

Term	Simple meaning	How to use it
AQI	An index that summarizes air quality using a defined scale and pollutant rules.	Show it as a public-facing air-quality indicator. Do not multiply AQI by time to calculate PM2.5 exposure.
PM2.5	Very small airborne particles, measured as a concentration such as µg/m³.	Use measured or appropriately estimated concentration for the weighted-average calculation.
Observation	A pollutant reading recorded at a location and time.	Keep the location, timestamp, units, provider, and method.
Forecast	A model's estimate of future conditions.	Label it as a forecast; do not mix it silently with a measurement.
Personal exposure estimate	A calculation combining available concentration values with time spent in different environments.	Explain which periods are included and what is unknown.

Suggested data sources to validate

Source	Possible use	Checks before relying on it
OpenAQ	Find monitoring locations and pollutant measurements from supported providers.	API key/access, city coverage, station coordinates, timestamp, units, update frequency, licensing/terms.
CPCB	Official Indian air-quality monitoring information and AQI context.	Current access method, station coverage, available fields, usage rules, data freshness.
Air-quality forecast provider	Broader hourly forecast context when direct measurements are unavailable.	Spatial resolution, forecast horizon, license, geographic coverage, and whether output is modelled.
Optional indoor sensor	Better evidence about the air in a particular room.	Sensor quality, calibration, placement, and cost; not required for the MVP.

Time-weighted average

Formula: average concentration = sum of (concentration × duration) divided by the total duration represented by usable values.

Example: A person has 2 hours at 100 µg/m³ and 8 hours at 25 µg/m³. The weighted average is ((2 × 100) + (8 × 25)) ÷ 10 = 40 µg/m³. This represents the time-weighted average across those 10 hours, assuming the example concentrations represent those periods.

Missing data and uncertainty rules

- Never replace missing measurements with zero.
- Never describe a forecast as a measurement.
- Show total tracked time separately from time covered by usable data.
- Exclude unknown intervals from the weighted average and clearly show the covered duration.
- Use compatible concentration units and reject invalid or negative duration values.
- Do not infer precise indoor conditions from outdoor station readings.
- Do not label the average as a medical-risk score or exact inhaled dose.

- Record where each value came from and when it was observed.

How to find the main contributor

For each period with usable data, calculate its weighted contribution: concentration × duration. To explain contribution to the overall average, divide that period's weighted amount by the sum of all weighted amounts. Also show concentration and duration separately, because a short high-concentration period and a long moderate-concentration period can affect the total differently.

6. Technical architecture

Choose the simplest stack the team can implement and explain.

Suggested stack

Part	Suggested technology	Why it is used
Frontend	React + Vite	Builds the pages and user interactions.
Styling	Tailwind CSS	Makes responsive styling quicker.
Charts	Recharts	Displays daily and weekly trends.
Map (optional)	Leaflet + OpenStreetMap tiles	Shows saved places and station locations when useful.
Backend	Python + FastAPI	Connects the UI to data sources and calculation logic.
Validation	Pydantic	Checks that API inputs and outputs have the expected shape.
Storage	PostgreSQL if persistence is needed	Stores saved places, activities, and summaries.
Hosting	A qualifying AWS service or qualifying AWS open-source tool	Satisfies the event's stated AWS eligibility rule.

Data flow in plain English

1. User Adds places and activity times
2. React frontend Shows screens and sends requests
3. FastAPI backend Validates input and calls the data provider
4. Data provider Returns available pollutant values and timestamps
5. Exposure calculation Matches usable values to time periods and calculates summary
6. Frontend result Displays value, coverage, source, uncertainty, and action

Possible repository structure

air-exposure-companion/

frontend/ # React pages and UI components

backend/

app/

main.py # FastAPI entry point

routes/ # API endpoints

services/ # Air-quality provider integration

exposure/ # Calculation and data-quality rules

models/ # Request/response and storage models

tests/ # Backend and calculation tests

docs/ # MVP notes, API contract, demo script

.env.example # Names of required variables; no real secrets

README.md # Setup, run, test, and deploy instructions

Agree on the API contract early

Before the frontend and backend are built separately, agree on the shape of one response. For example, an air-quality result should include pollutant name, value, unit, timestamp, source, data type, and location if available. An exposure summary should include weighted average, tracked duration, usable-data duration, and a list of included periods. This prevents mismatched assumptions during integration.

7. Step-by-step project completion plan

Use these milestones in order. The team can work in parallel after the data format and API contract are agreed.

Milestone 0 — Kickoff and scope lock

Owner: All three members

- Read this plan together and agree on the first city and target user.
- Choose the exact core demo story: routine → exposure estimate → contributor → action.
- Confirm hackathon submission requirements, deadline, AWS eligibility, and demo format.
- Create the Git repository, issue board, communication channel, and shared task list.

Completion check: Done when: every member can explain the MVP in one sentence and the must-build list is frozen.

Milestone 1 — Validate real data

Owner: Member A leads; B supports

- Check available monitoring locations in the chosen city.
- Fetch a small sample of PM2.5 observations and inspect value, unit, time, location, and source.
- Check whether several hours of usable historical data are available for a demo.
- Write down gaps and decide how the app will label measurements, forecasts, and estimates.

Completion check: Done when: the team has a reproducible sample and a written decision on which data source to use. If data quality is poor, change the source or narrow the geographic scope.

Milestone 2 — Define calculation and API contracts

Owner: A owns calculation; B owns contract; C reviews display needs

- Agree on the time-weighted average and how missing intervals are handled.
- Write example input and expected output for the exposure calculation.
- Define the API response fields and example JSON for air quality, activities, and exposure summary.
- Give C a stable example response for building the UI while the real backend is in progress.

Completion check: Done when: one hand-calculated example matches the agreed expected result and the API contract is written down.

Milestone 3 — Build the calculation module

Owner: Member A

- Implement the weighted-average calculation and usable-data coverage.
- Reject invalid durations and inconsistent units.
- Add tests for missing values, no usable data, one interval, and multiple intervals.
- Write a short explanation of what the result means and does not mean.

Completion check: Done when: tests pass and the module never treats missing data as zero.

Milestone 4 — Build the backend

Owner: Member B; A supports data rules

- Create the FastAPI application and health endpoint.
- Add endpoints for air-quality data, activities, daily exposure, weekly summary, and recommendations as needed.
- Integrate the selected data provider and handle timeouts, empty results, and stale data.
- Keep credentials in environment variables and provide an .env.example without secrets.

Completion check: Done when: endpoints return the agreed response format and show useful errors instead of crashing.

Milestone 5 — Build the frontend

Owner: Member C; B supports API integration

- Build Dashboard, My Exposure, My Places, and Recommendations/Reports.
- Make manual routine entry work.
- Show timestamps, sources, coverage, and measured/forecast/estimated labels.
- Use sample data only as a clearly labelled fallback or development fixture.
- Check mobile and desktop layouts.

Completion check: Done when: a user can move through the main screens and understands what the numbers mean.

Milestone 6 — Integrate end to end

Owner: All three; B coordinates

- Connect frontend requests to the backend.
- Run the same calculation example through the actual UI.
- Test no-data, stale-data, and provider-failure states.
- Fix mismatched field names, time zones, date boundaries, and unit errors.
- Remove placeholder values that could be mistaken for live data.

Completion check: Done when: a complete user journey works without manual code edits during the demo.

Milestone 7 — Add AWS and deploy

Owner: B leads; C supports frontend hosting; A verifies output

- Choose one qualifying AWS path after checking the event rules.
- Set environment variables and secrets securely.
- Deploy the app or use a qualifying AWS open-source tool in a real, explainable way.
- Check the deployed URL, API connectivity, logs, and error behaviour.
- Review expected charges and remove unused resources.

Completion check: Done when: the deployed or qualifying build works and the team can explain AWS's role in under 20 seconds.

Milestone 8 — Validate, document, and rehearse

Owner: All three

- Run calculation and end-to-end tests.
- Ask a few people unfamiliar with the project to try the main journey.
- Record a three-minute demo showing the real working flow.
- Prepare a simple architecture diagram and list data limitations.
- Write the AWS Builder Center build post and check all submission requirements.

Completion check: Done when: the demo recording is clear, the product works from start to finish, and submission assets are ready.

8. Team workflow and integration rules

Git and branches

- Keep one shared repository and a stable main branch.
- Use short feature branches, such as feature/exposure-calculation or feature/dashboard.
- Open a pull request or ask another teammate to review important changes before merging.
- Commit small, working changes rather than one huge final commit.
- Do not commit API keys, passwords, real user location histories, or private configuration.

Issue format

Every task should say what needs to be done, who owns it, how to test it, and what counts as complete. Example: “As a user, I can enter two activities and see the correct weighted PM2.5 average. Test: 2 hours at 100 and 8 hours at 25 gives 40 µg/m³.”

Definition of done for each feature

- The feature works in the agreed user journey.
- Inputs are validated and errors are handled.
- The displayed values match the backend calculation.
- Measured, forecast, estimated, and unavailable data are clearly distinguished.
- The code is committed and another teammate can run it using the README.
- A basic test or manual verification step exists.

How to avoid integration problems

- Agree on field names, units, timestamps, and time zones before parallel development.
- Use one shared example response file or API contract.
- Do not let the frontend invent its own exposure calculation if the backend owns the calculation.
- Keep one known test scenario that everyone uses to verify the result.
- Merge working pieces early rather than waiting until the final day.

Decision rule when the team disagrees

Prefer the option that is scientifically honest, smaller to implement, easy to test, and easy to demonstrate. If two options provide similar user value, choose the simpler one. Record postponed ideas instead of repeatedly reopening the scope.

9. Testing and validation

Exposure calculation test cases

Case	Input	Expected behaviour
Basic weighted average	2 h at 100 µg/m³; 8 h at 25 µg/m³	Average = 40 µg/m³ over 10 hours.
One interval	1 h at 50 µg/m³	Average = 50 µg/m³.
Missing interval	A time period has no valid measurement	Do not use zero; exclude from usable average and show coverage gap.
No usable data	All values missing or invalid	Show “not enough data”; do not show a numeric average.
Invalid duration	End time is before start time	Reject or ask user to correct the activity.
Different units	Values are not expressed in the same concentration unit	Convert safely if supported or reject; never combine blindly.
Partial day	Only 6 of 12 tracked hours have usable data	Show the average for covered time and show 6/12 hours coverage.

API tests

- Provider returns valid data.
- Provider returns no stations or no readings.
- Provider is slow, unavailable, or returns an error.
- Reading timestamp is old.
- Response contains unexpected units or missing fields.
- API key is absent or invalid.
- Requests for one city do not accidentally show results from another city.

Usability checks

- Can a first-time user understand the dashboard without a spoken explanation?
- Can the user tell the difference between AQI and PM2.5 concentration?
- Can they see how many hours are covered by usable data?
- Can they tell which values are measured, forecast, or estimated?
- Can they edit a routine and see the summary update?
- Can they find the data source and timestamp?

Do not overclaim

A working prototype does not prove that the product reduces disease or that its exposure estimate equals a laboratory measurement. For the hackathon, demonstrate correct calculations, transparent data handling, useful recommendations, and a credible user workflow. Treat real-world health impact as a goal to evaluate later.

10. AWS eligibility, deployment, and cost safety

The event text supplied by the team says a project must either use at least one qualifying AWS open-source tool or be deployed on AWS to be eligible for prizes. Re-check the official rules before submission because the exact requirements and services can change.

Choose one clear AWS path

Path	Possible approach	What to explain to judges
Deploy the application	Host the frontend and backend on suitable AWS services, selected after checking the current free-tier terms and pricing.	Which component runs on AWS, how the frontend reaches the API, and how secrets are handled.
Use a qualifying AWS open-source tool	Select a tool explicitly accepted by the event rules and use it as a real part of the development or runtime workflow.	What the tool does and where it is used; do not include it only as a name in the README.

Keep AWS simple

- Do not add a database or serverless workflow unless the product needs it.
- Use environment variables or a suitable secret-management method for credentials.
- Set budgets or billing alerts where available, and remove resources you no longer need.
- Check current region availability, pricing, and free-tier conditions before deploying.
- Verify the final URL and API response before recording the demo.

What AWS should contribute

AWS should provide a real part of the product's hosting, API execution, storage, or another qualifying function. In the demo, point to the actual deployed component or show how the qualifying open-source tool is used. Avoid claiming that AWS itself makes the pollution estimate more accurate; the data and calculation determine that.

11. Three-minute demo and judging strategy

The judges will not have time to inspect every screen. Tell one clear story: a person has a routine, the app estimates the pollution conditions across the tracked period, the app explains a contributor, and the person receives an action they can take.

Time	What to show	What to say
0:00–0:25	Problem and target user	“AQI tells me about an area. It does not explain the pollution conditions across my own routine.”
0:25–0:55	Add or open a sample routine	Show home, commute, and college/office activities. State if the routine is a demo example.
0:55–1:35	Exposure timeline and calculation	Show PM2.5 values, duration, weighted average, tracked time, and data coverage.
1:35–2:05	Main contributor and action	Explain what the data supports and show a practical recommendation.
2:05–2:35	Data quality and AWS	Show source/timestamp/uncertainty and briefly explain the actual AWS component.
2:35–3:00	Impact and closing	Explain who benefits, what works today, and the next validation step.

Demo preparation

- Use a reliable, pre-tested scenario. Do not depend on an unpredictable live API call as the only way to demonstrate the product.
- If using sample data, label it clearly and explain the calculation still runs in the real app.
- Record the screen at a readable size and make text large enough to see.
- Do not spend 30 seconds on login or setup.
- Show the end-to-end product, not a slide deck claiming future features.
- Keep one backup recording in case the live deployment is slow.

Blog post outline

- The real problem and who experiences it.
- Why ordinary AQI is not the same as personal exposure.
- How the team chose and validated data sources.
- How the exposure calculation works and what its limitations are.
- The technology stack and actual AWS contribution.
- What worked, what was difficult, and what the team would improve next.

12. Risks and distraction guardrails

Risk	Why it matters	Response
Poor station coverage	A station may be far from a saved place or report too infrequently.	Validate coverage first; show distance, time, and uncertainty; narrow the MVP area if needed.
False precision	A numeric score can look scientific even when the inputs are weak.	Show tracked time and usable-data coverage; do not claim exact inhaled dose.
Mixed data types	Measurements, forecasts, and estimates are not interchangeable.	Store and display the data type for every value.
Scope creep	Too many features leave the core journey unfinished.	Freeze must-build scope; put extras on a later list.
API failure	A live demo may fail due to rate limits or provider downtime.	Handle errors; test a labelled sample-data fallback.
Location privacy	Location history can reveal sensitive routines.	Make tracking optional, collect less, and offer deletion.
AWS cost surprise	Resources can incur charges after promotional credits or free usage end.	Check pricing, set alerts, remove unused resources.
Weak demo	Judges may not understand the product's unique value.	Show one end-to-end story with working calculations and a clear AWS role.

Distraction test

Before accepting a new feature, ask: Does it improve the quality of the exposure estimate? Does it help the user understand or reduce exposure? Is it needed for eligibility? Does it make the three-minute demo stronger? If all answers are no, do not build it now.

Features most likely to distract

- AI chatbot before reliable data and calculation.
- Diet plans and workout generators unrelated to exposure reduction.
- Precise street-level route rankings without suitable route-level pollution data.
- Multiple cities before the first city works.
- Custom ML models before a useful baseline and evaluation data exist.
- Visual effects and animations before error states and calculations are correct.

13. Final completion and submission checklist

Product and data

- ■ One city selected based on usable data coverage.
- ■ Source, timestamp, units, and data type shown for air-quality values.
- ■ Manual places and activity entry work.
- ■ Daily exposure calculation has automated tests.
- ■ Missing and stale data are handled honestly.
- ■ Tracked duration and usable-data duration are visible.
- ■ Recommendations are evidence-based and explain their reason.
- ■ No sample values are presented as live readings.

Team and engineering

- ■ Each of the three members owns a clear workstream.
- ■ API contract and data fields are documented.
- ■ Repository has setup instructions and an example environment file.
- ■ Frontend and backend work together end to end.
- ■ Core errors are tested.
- ■ Location tracking is optional and user data can be removed.

Hackathon

- ■ Official eligibility and deadline checked.
- ■ Qualifying AWS usage or deployment verified.
- ■ AWS costs and resources reviewed.
- ■ Three-minute demo recorded and checked.
- ■ Architecture and data limitations can be explained simply.
- ■ AWS Builder Center write-up prepared and linked in the submission.
- ■ Submission form and any student verification requirements completed.

14. Official references and useful starting points

Use these links to validate current documentation, API requirements, and event rules. Availability, access terms, and service pricing can change; verify them before implementation.

Resource	Link	Why it matters
OpenAQ API quick start	https://docs.openaq.org/using-the-api/quick-start	Starting point for checking API access and retrieving air-quality measurements.
OpenAQ Explorer	https://explore.openaq.org/	Explore available monitoring locations and data coverage.
CPCB AQI portal	https://airquality.cpcb.gov.in/AQI_India/	Indian official air-quality information and station context.
India Open Government Data — real-time AQI	https://www.data.gov.in/catalog/real-time-air-quality-index	Dataset catalogue; verify current access method and fields.
Open-Meteo Air Quality API	https://open-meteo.com/en/docs/air-quality-api	Air-quality forecast documentation; check resolution and terms.
AWS Free Tier	https://aws.amazon.com/free/	Current free-tier details; verify eligibility and conditions.
AWS Pricing Calculator	https://calculator.aws/	Estimate service costs before deployment.
AWS Builder Center	https://builder.aws.com/	Builder profile and publishing resources mentioned in the event information.
FastAPI documentation	https://fastapi.tiangolo.com/	Backend framework documentation.
React documentation	https://react.dev/	Frontend framework documentation.
Tailwind CSS documentation	https://tailwindcss.com/docs	Styling documentation.
WHO air pollution resources	https://www.who.int/health-topics/air-pollution	Background information on air pollution and health.

Final note: This document is a build plan, not a guarantee of data availability, a medical tool, or a confirmation of current hackathon eligibility. Validate the chosen data source and official event rules before implementation.